

# Implementation of K-Nearest Neighbours (K-NN) in Python

## Aim

To implement the **K-Nearest Neighbours (K-NN)** algorithm in Python for classifying a given dataset and predicting the class of new samples.

## Apparatus Required

- Google Colab

## Theory

K-Nearest Neighbours (K-NN) is a **supervised machine learning algorithm** used for classification and regression. It classifies a new data point by finding the **K nearest training samples** based on distance (usually Euclidean distance). The majority class among these neighbors is assigned to the new sample.

## Advantages:

- Simple and easy to implement.
- No training phase required.
- Works well for small datasets.

## Disadvantages:

- Slow for large datasets.
- Sensitive to the choice of K.
- Requires feature scaling.

## Algorithm

1. Import the required libraries.
2. Load the dataset.
3. Split the dataset into training and testing sets.
4. Standardize the feature values.
5. Create the K-NN classifier with a chosen value of K.
6. Train the classifier using the training data.

7. Predict the class labels for the test data.
8. Calculate the accuracy and display the confusion matrix.

Code:

```
import pandas as pd

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix


# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target


# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)


# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


# KNN Model
```

```
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
```

```
# Prediction
```

```
y_pred = knn.predict(X_test)
```

```
# Results
```

```
print("Predicted Classes:")
```

```
print(y_pred)
```

```
print("\nAccuracy:")
```

```
print(accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

## OUTPUT

```
*** Predicted Classes:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy:
1.0

Confusion Matrix:
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

**Result :** The **K-Nearest Neighbours (K-NN)** algorithm was successfully implemented in Python using the Iris dataset. The classifier predicted the test samples with high accuracy, demonstrating the effectiveness of the K-NN algorithm for classification tasks.